

Wake-up

La task Wake-up, alla base, è identica a quella del Broadcast con un'unica differenza sostanziale:

La presenza di più nodi sorgente.

Più formalmente dato un insieme E di agenti avremo

Un insieme $W \subseteq E$ di agenti 'svegli' che vogliono 'svegliare' i restanti nodi all'interno di E/W

Come già detto questa task è una generalizzazione del Broadcast in quanto

il Broadcast è un Wake-up con un solo initiator

Assunzioni e restrizioni

La task del Wake-up opererà sotto le assunzioni:

- Total reliability
- Bidirectional links
- Connectivity

Le assunzioni sono identiche a quelle del Broadcast con l'unica differenza che non assumiamo Unique Initiator

Stati possibili

Esattamente come in Broadcast avremo 3 stati possibili:

- Init
- Idle
- Done

Hanno la stessa definizione che hanno in Broadcast

Configurazione iniziale e finale

Analizziamo ora le possibili configurazioni iniziali e finali

- Configurazione iniziale:

Un insieme $W \subseteq E$ agenti in Init e gli agenti in E/W nello stato Idle $\rightarrow 2^n$ possibili configurazioni iniziali

$|E|=n$

- Configurazione finale:

$\forall x \in E$ x si trova nello stato Done $\rightarrow$ 1 sola configurazione finale

Protocollo WFlood

Il protocollo WFlood non è altro che il protocollo Flood adattato all'evenienza di avere più nodi initiator

Analisi della msg complexity

Per quanto riguarda l'analisi della msg complexity questa viene eseguita calcolando quanti msg invia ogni nodo:

- Se $x \in W$ Allora x invia I ad $N(x)$
- Se $x \notin W$ Allora x invia I ad $N(x) \setminus \{father\}$

Sapendo questo possiamo calcolare

$$Msg(*) = \sum_{x \in W} N(x) + \sum_{x \notin W} N(x) - 1 = \begin{cases} 2m & \text{se } W=V \\ 2m-n-1 & \text{se } |W|=1 \end{cases}$$

Essendo il problema così simile al Broadcast è facile far vedere come valga il Lower bound $Msg(*) = \Omega(m)$

Wake-up su grafi specifici

Modificando la topologia del grafo è possibile trovare un'ulteriore Lower bound

Assunzioni

Per questo nuovo Lower bound lavoreremo su grafi:

- Complet
- Unique labeling $\leadsto$ Tutti i nodi hanno un'etichetta unica conosciuta da tutti gli altri nodi

Lower bound

All'interno di sistemi rappresentabili mediante grafi completi e con Unique labeling vale il seguente lower bound sulla msg complexity:

$$Msg(Wake\ up) = \Omega(n \log n)$$

Dim

Per dimostrare questo Lower bound verrà creato un gioco tra le entità, che seguiranno il protocollo, e l'avversario che cercherà di inviare più messaggi possibili.

L'avversario potrà usufruire solo di queste 4 azioni:

1) Sceglie le etichette dei nodi

2) Decide quando e quali entità iniziano spontaneamente l'esecuzione del protocollo

3) Decide quando un messaggio inviato verrà ricevuto

4) Decide l'accoppiamento fra archi ed etichette

→ Ovviamente dovrà sempre agire sotto le assunzioni del graf

Siano $e_1, \dots, e_k$ gli archi incidenti con x e $l_1, \dots, l_k$ le etichette degli archi usati da x . Quando x esegue "Send to l " ed l non è ancora stato associato ad un arco allora l'adv. sceglie a quale degli archi non usati verrà associata l'etichetta

Prima di mostrare la strategia dell'adv. diamo ora questa definizione:

Def

Due sottoinsiemi $A, B \subseteq V$ sono connessi al tempo t se

Almeno un messaggio è stato inviato da $a \in A$ verso $b \in B$ o viceversa al tempo t

Mostriamo ora la strategia dell'avversario

1) Inizialmente l'avversario sveglia una sola entità s che inizierà l'esecuzione del protocollo

Quando s deciderà di mandare il messaggio alla porta l , l'avversario sveglia un'altra entità y ed assegnerà l all'arco fra s ed y

farà in modo che il messaggio di s verso y venga ritardato finché anche y decide di trasmettere il messaggio su l

A quel punto l'avversario assegnerà ad l l'arco (x, x) e sarà in modo che i due messaggi vengano ricevuti contemporaneamente

2) Sia $A \subseteq V$ t.c. $A = \{x_1, \dots, x_k\}$

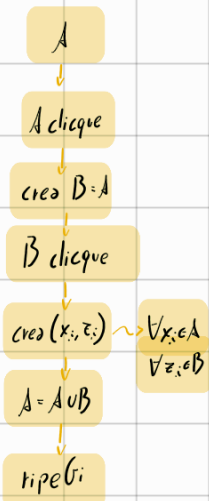
Vediamo ora che succede quando $\forall x_i$ invia il msg verso l'etichetta a

• Se $\exists x_i \in A$ t.c. x_i è sveglio e x_i non ha ancora usato l'arco (x_i, x_i) Allora l'ADV assegna l'etichetta a ad (x_i, x_i)

• Se $\nexists x_i \in A$ sveglio e con un arco non usato Allora:

1) L'ADV creerà un nuovo set $B = \{z_1, \dots, z_k\}$ di nodi inattivi t.c. B è una copia esatta di A e come tale B avrà la stessa identica esecuzione di A

Questo sarà la logica dell'ADV V scambio di msg



ii) L'ADV ora assegnerà l'etichetta a all'arco (x_i, z_i) e rallenterà il msg

iii) Avendo B la stessa storia di A allora $\exists$ un istante in cui z_i non può più mandare messaggi all'interno di B

Per questo z_i manderà il msg sull'etichetta b

A quel punto l'ADV assegnerà b all'arco (z_i, x_i) e solo allora l'ADV sarà arrivare il msg sull'arco (x_i, z_i)

Analizziamo ora il costo di tutte queste operazioni

Sia lo stage la fase appena prima l'interconnessione fra i set A e B

Definiamo

- $Active(i)$ i nodi attivi all'interno dello stage i
- $New(i) = Active(i) - Active(i-1)$ i nodi svegliati dall'ADV nello stage i

Osserviamo ora che $|New(i)| = |Active(i-1)|$

Chiamiamo ora $\mu(i-1)$ il numero di msg scambiati in $Active(i-1)$ prima dell'attivazione dei nuovi nodi $New(i)$

Assicure i nodi $New(i)$ abbiano la stessa stessa storia anch'essi dovranno inviare $\mu(i-1)$ msg

Quindi prima di passare allo stage in cui i due set comunicano fra loro vengono inviati un totale di

$$2\mu(i-1)$$

Prima di passare allo stage i il nodo $x \in New(i) \cup Active(i-1)$ deve aver ricevuto o inviato un msg da tutte le entità in $New(i) \cup Active(i-1)$

Analizziamo ora i 2 casi:

- Se $x \in New(i)$:

Vuol dire che x è stato svegliato all'interno dello stage(i) e quindi bisogna mandare e/o ricevere

$$|Active(i-1)| \text{ msg}$$

- Se $x \in Active(i-1)$:

Vuol dire che x deve comunicare solo con $New(i)$ avendo già comunicato con i nodi in $Active(i-1)$ negli stage passati

$$\text{Per cui anche in questo caso deve mandare e/o ricevere } |New(i)| = |Active(i-1)| \text{ msg}$$

Per cui prima di passare allo stage $(i+1)$ bisogna inviare

$$\mu(i) \geq 2\mu(i-1) + |Active(i-1)|$$

Essendo che le entità sveglie raddoppiano ad ogni stage allora

$$|Active(i)| = 2^i$$

Ci osservando che $\mu(0) = 0$ abbiamo che

$$\mu(i) \geq 2\mu(i-1) + 2^{i-1} \geq i \cdot 2^{i-1}$$

Per cui l'ADV deve inviare almeno

$$\mu(\log n) \geq n \log n \text{ messaggi}$$